

第八章 智慧农业系统构建

尹江津

信息学院

Q Q:

1961015633

微 信:

15674996078

8.1 智慧农业系统生命周期概述

8.2 智慧农业系统定义时期

8.3 智慧农业系统开发时期

8.4 智慧农业系统维护时期

第一部分——智慧农业 系统的生存周期

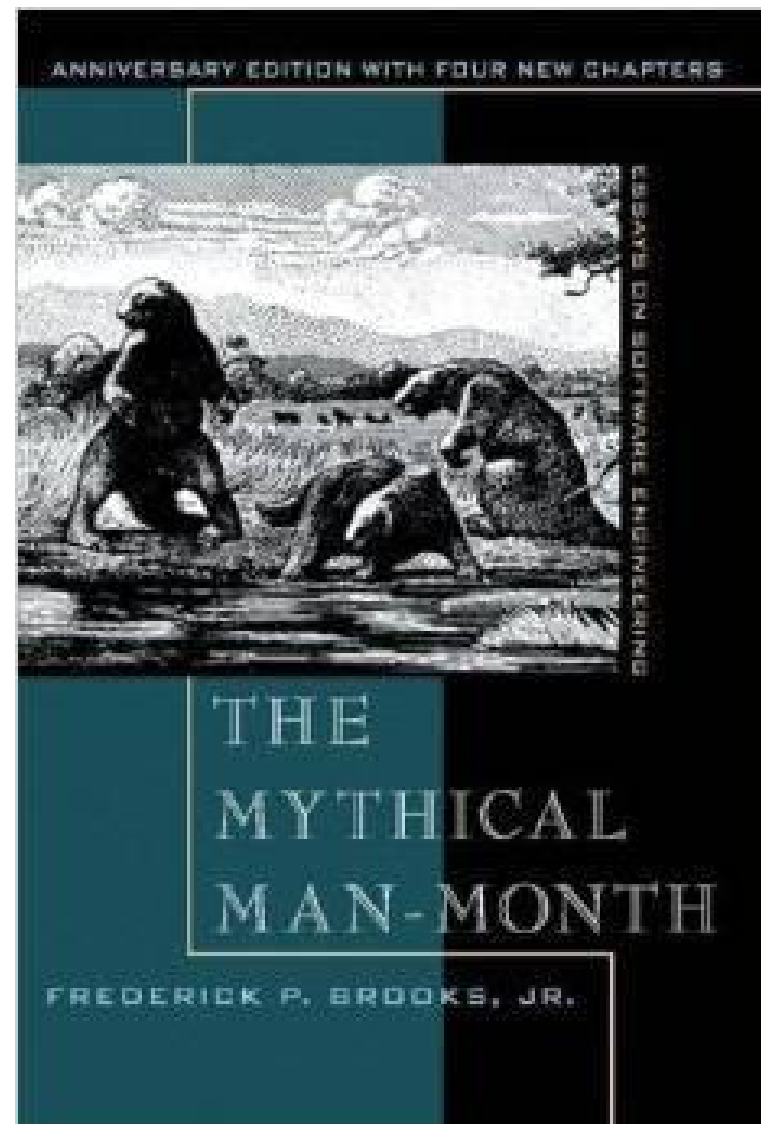
有这样一个项目需要12个人月，即2个人参与6个月就能完成该任务。项目实施两个月后，管理者发现，这两个月才完成原本一个月需要完成的任务。为了按时完成项目，管理者准备增加人手，请问最少加几人才能按时完成？（ ）

- ☐ A 2人
- ☐ B 3人
- ☐ C 4人
- ☒ D 都不行

Brooks布鲁克斯法则

- ❑ 《The Mythical Man-Month》，中文名《人月神话》，是由Frederick P. Brooks Jr.所著，是关于软件工程和项目管理的经典著作，是描述大规模软件开发难度的一本划时代的书。
- ❑ 书名中的“Man-month”是指人力月或人月，是一个衡量项目工作量的单位，表示一个人工作一个月的工作量。
- ❑ 书中提到的**Brooks法则**：即在项目已经落后的情况下增加人员只会使项目更加落后。

因为新成员需要时间来学习代码、了解设计，而现有成员必须花费更多的时间来培训和协调。



软件危机

- ❑ 对软件开发成本和进度估计常常不准确。开发成本超出预算，实际进度比预定计划拖后。
 - ❑ 用户对“已完成”系统不满意的现象经常出现。
 - ❑ 软件产品质量往往靠不住，Bug一大堆。
 - ❑ 软件可维护程度非常低。
 - ❑ 软件通常没有适当的文档资料。
 - ❑ 软件的成本不断提高。
 - ❑ 软件开发生产率的提高赶不上硬件发展和人们需求增长。
-

软件工程

- ❑ **软件工程** (software engineering)：是用工程、科学和数学的原则与方法开发、维护计算机软件的有关技术及管理方法。
- ❑ **软件工程**这个词是北大西洋公约组织科学委员会1968年在联邦德国举行的一次会议中提出的。



本章以计算机软件技术
为主来说明智慧农业系
统的构建方法。

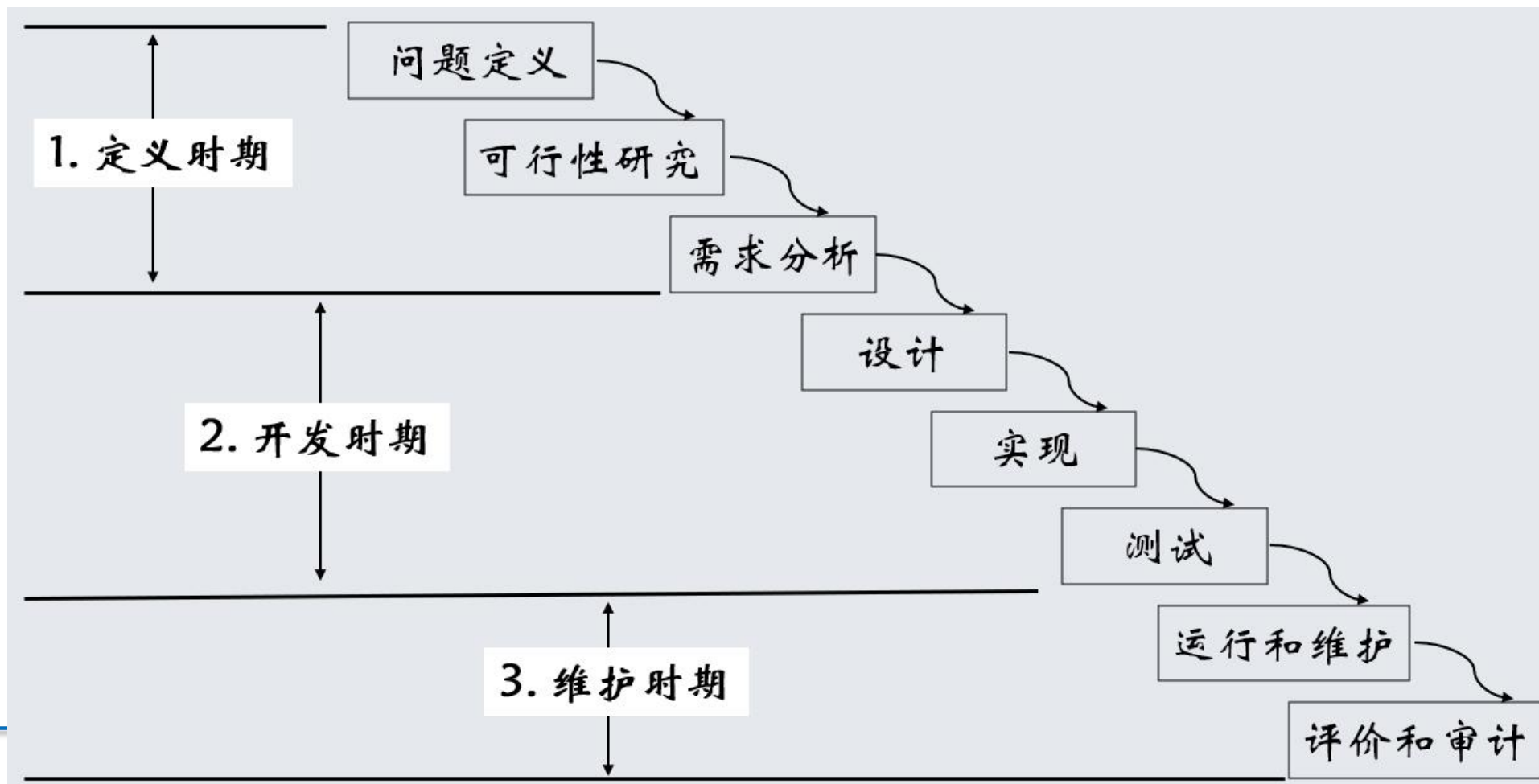
1. 智慧农业系统的生命周期

生命周期，亦称生存期，就是指一个对象的生老病死的全过程。

- 一个智慧农业系统从孕育、诞生、成长、成熟、衰亡的发展过程称为该智慧农业系统的**生命周期或生存期**。
- 智慧农业系统生命周期由**三个时期组成：系统定义时期、系统开发时期、系统维护时期**。

1. 智慧农业系统的生命周期

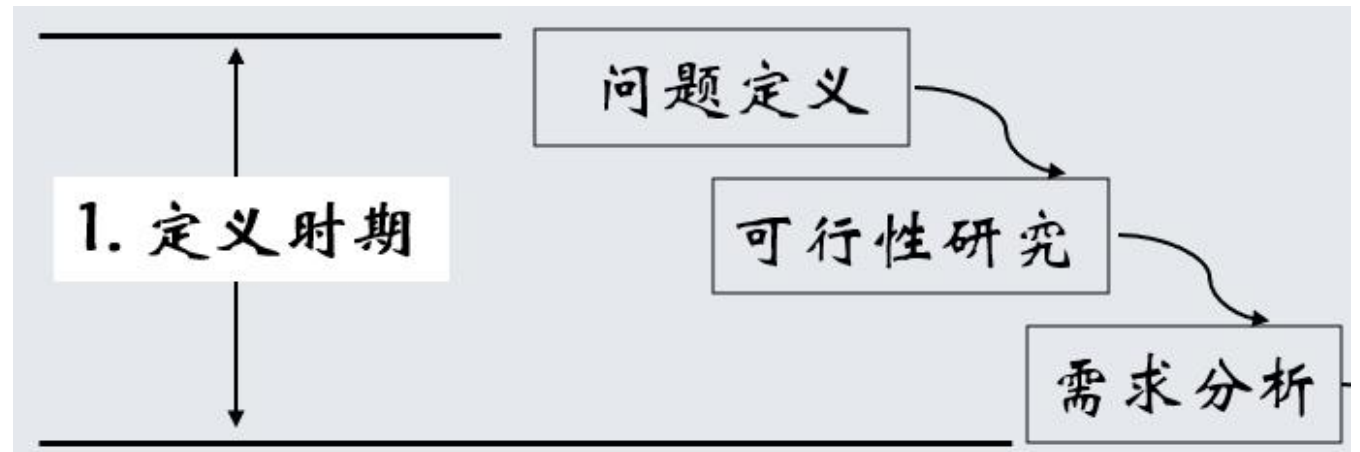
- 智慧农业系统生命周期由三个时期的组成：**定义时期**、**开发时期**、**维护时期**。
- 每个时期又可以划分为若干阶段。



1. 智慧农业系统的生命周期

1. 定义时期

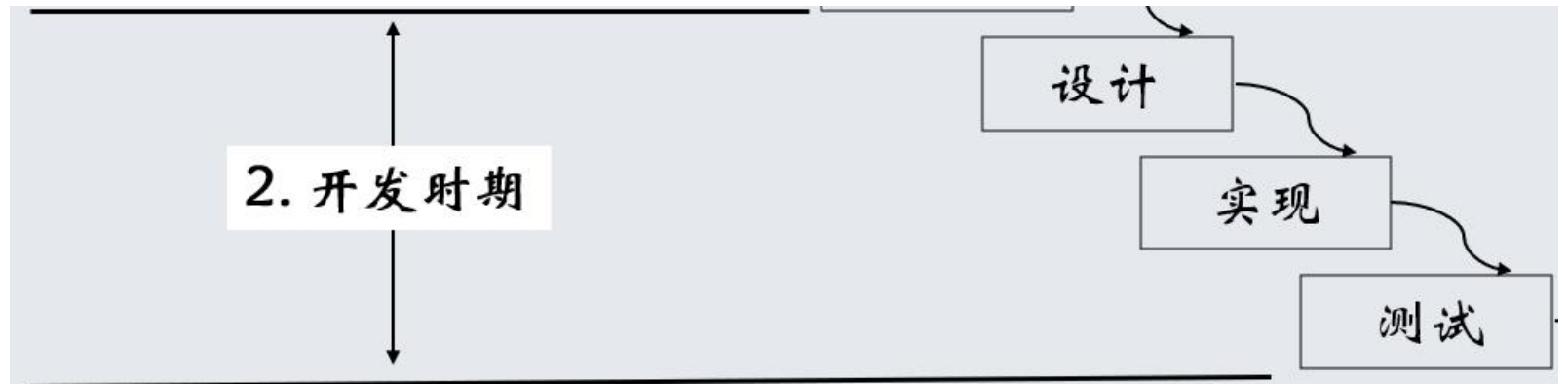
- 定义时期的主要任务是解决“**做什么**”的问题。
 - 确定智慧系统工程的**总目标和可行性**；
 - 导出实现工程目标应使用的策略及系统必须完成的功能；
 - 估计完成工程所需要的资源和成本；
 - 制定工程进度计划。
- 该时期的工作也就是我们常说的**系统可行性分析**和**系统需求分析**。
- 定义时期通常被分为三个阶段：**问题定义**、**可行性研究**、**需求分析**。



1.智慧农业系统的生命周期

2. 开发时期

- 开发时期的主要任务是解决智慧农业系统“**如何做**”的问题。
- 具体是设计和实现在定义时期的软件、硬件。
- 通常由**设计（概要设计和详细设计）**、**实现（编码实现）**和**测试**三个阶段组成。



1.智慧农业系统的生命周期

3. 维护时期

- **维护时期**是智慧农业系统**已经交付用户使用后**，为了使得系统软硬件持久地满足用户的需求，而不断做出修改和完善的过程，并对系统使用效果做出评价和审计。
- 通常由**运行和维护**和**评价和审计**两个阶段组成。



第二部分——软件定义时期

系统可行性分析与需求分析

2.1 系统可行性分析

- 开发一个基于计算机的系统通常都受到资源（人力、财力、设备等）和时间上的限制，可行性分析主要从经济、技术、法律等方面分析所给出的解决方案是否可行，能否在规定的资源和时间的约束下完成。
 - 可行性分析目的**：通过调查研究全面分析与系统有关的因素，各种可能方案，并对各个方案的经济效益进行分析，最后评选最优方案，为决策者提供决策依据。
 - 可行性分析的任务**：在做出决策之前，全面论证系统开发的必要性、可能性、有效性和合理性。
 - 可行性分析主要包括**：**经济可行性、技术可行性、环境与操作可行性。**
-

2.1 系统可行性分析

1. 经济可行性分析

经济可行性分析主要从**费用**和**经济效益**两方面进行。

- **费用估计**：费用分为智慧农业系统**开发建设费用**和**投入使用后的运行费用**。
费用估计是对智慧农业系统开发和运行的整个过程进行**总的费用估算**。
 - **经济效益估算**：若系统设计合理，则在系统投入运营后会取得一定的经济效益和社会效益。在可行性研究阶段，系统尚未建成，因此**只凭借系统分析人员的经验进行预测**。
-

2.1 系统可行性分析

1. 经济可行性分析——开发运行费用

通常智慧农业系统的**开发运行费用**主要包括以下开支

□开发费用

- 农业设施、计算机机房费用，包括土地租赁、办公场地和建设的费用等；
- 计算机及其外围设备的购置费用，包括全部硬件设备，如传感器、服务器、电脑、网络设备 etc；
- 系统开发和软件的费用，包括开发或购买软件费用、技术咨询费用等；
- 系统调试和安装费用，包括安装、调试系统的一切硬件和软件费用。

□运行费用

- 培训费，包括与系统运行有关的一切人员技术培训和业务培训应付的费用；
- 资料费，包括购置培训材料、学习资料、书籍、复印费等；
- 人员工资，智慧农业系统使用、操作人员的工资；
- 消耗品费用，包括系统应用的一切消耗品费用，水、电、农资等各类生产资料的费用；
- 技术服务性费用，包括调研、研讨、差旅等费用。

2.1 系统可行性分析

1. 经济可行性分析——经济效益估算

经济效益估算分为直接经济效益和间接经济效益。

□ 直接经济效益

- **可以直接用数字表达出来。**如节省人力、财力、时间，增加产量、提高生产效率等，可以通过投资回报率、投资回收期、经济效益、纯收入等直接用数字描述。

□ 间接经济效益

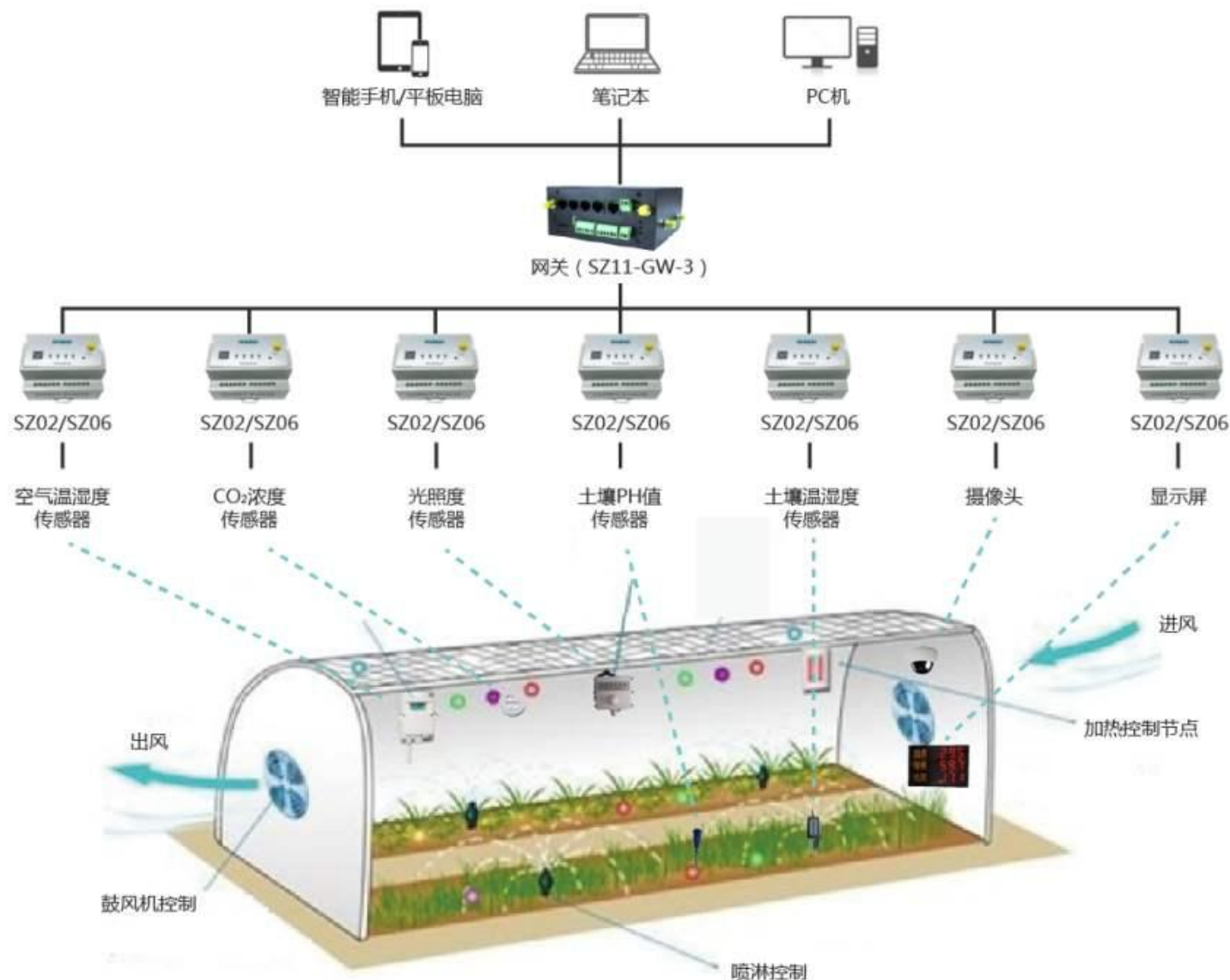
- **即难以用数字直接表达出来，也称社会效益。**如解放繁重的体力劳动和脑力劳动，增加信息获取的准确度、及时性，改善用户服务、提高信誉和用户满意度等。
-

2.1 系统可行性分析

1. 经济可行性分析

智能温室成本

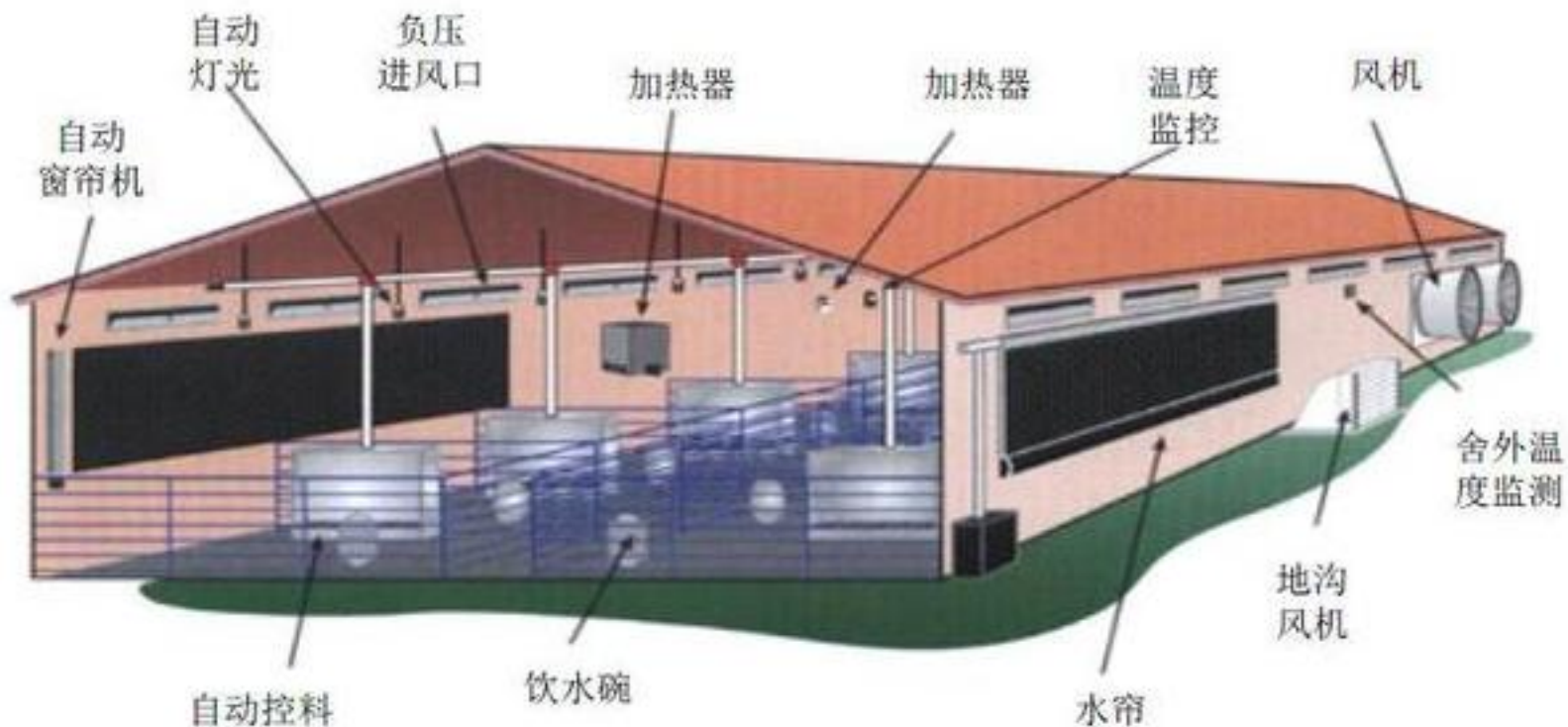
- 玻璃大棚300~400元左右
每平方米
- 阳光板大棚200~300元左右
每平方米
- 薄膜大棚80~160元左右
每平方米



2.1 系统可行性分析

1. 经济可行性分析

猪场成本：选址、建设、引种、设施、饲喂、环保排污、疾病防护等



2.1系统可行性分析

2. 技术可行性分析

技术可行性是指在特定的时间、资源和条件下，利用现有的科学技术知识和能力，实现项目目标的可能性。它主要关注的是在技术层面上能否解决预期的问题或者达成设定的目标。主要内容如下：

- **开发技术的冒险性。**指系统能否实现所要求的功能和性能，成功的可能性多大。
 - **现有资源的可用性。**主要从三个方面进行考虑：人力资源（团队成员是否具备相应技能和经验）、硬件资源（是否拥有满足系统运行需求的服务器、网络设备等基础设施）和软件资源（是否拥有合适的开发工具、操作系统、数据库管理系统等）。
 - **系统确定的开发技术。**考察选定的开发技术是否成熟并足以支撑新系统的构建。对于系统设计方案中的关键技术环节，如大数据处理、人工智能算法、云服务集成等，都需要逐一深入分析其在实际应用中的可行性、成熟度以及稳定表现。
-

2.1 系统可行性分析

3. 环境与操作的可行性分析

□环境包括**社会环境**和**自然环境**。

- **社会环境可行性**是在特定环境下对项目的开发与实施带来的社会影响。社会环境可行性分析包括社会因素的可行性、法律可行性、社会推广可行性、使用可行性等。例如，该系统的开发将不会侵犯任何个人、集体、国家的利益，也不会违反国家的政策与法律。
- **自然环境可行性分析**包括环境保护、污染物排放等对智慧农业系统的要求。建设智慧农业系统需要考虑国家有关环保的相关法律，包括环境保护法、水污染防治法、大气污染防治法、固体废物污染环境防治法、环境噪声污染防治法、海洋环境保护法等。

□**操作可行性**侧重于评估用户和系统操作人员在日常使用和维护系统时的便利性、效率和安全性。主要包括：用户界面设计、操作流程、系统管理与维护等。

2.1 系统可行性分析

- 可行性研究报告
 - 引言；
 - 可行性研究的前提；
 - 对现有系统的分析；
 - 所建议的系统；
 - 可选择的其他系统方案；
 - 投资及效益分析；
 - 社会因素方面的可行性；
 - 结论。

2.2 智慧农业系统的需求分析

- 《神雕侠侣》的客户**杨过**想要一款集美丽、智慧于一身、仙气飘飘的**小龙女**。理想很丰满，可是现实很骨感。最后交付造出来的产品与客户想要的相去甚远，**杨过万念俱灰，跳崖轻生，反倒练成绝世武功。**



需求分析



原型设计



产品开发



验收交付



验收交付



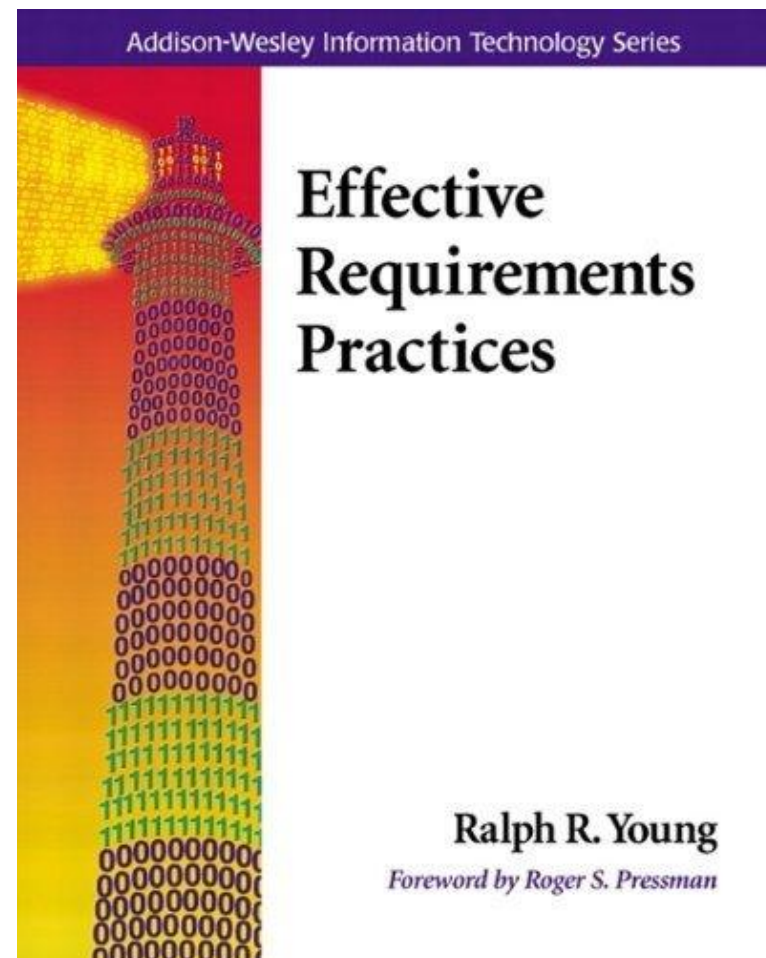
验收交付

需求失真 是罪魁祸首！

2.2 需求分析

- 理解问题的需求是**软件工程师所面对的最困难的任务之一**。
- 《Effective Requirements Practices》是一本探讨软件开发过程中需求工程领域实践的专著。书中写道，一个客户走进你的办公室说：“我知道你认为你理解我说的是什么，但你并不理解的是我所说的并不是我想要的。”
- 书中把这个现象比喻为一个**“最恐怖的噩梦”**，即客户指出尽管工程师自认为已经理解了他们的需求，但实际上并未把握住客户真正意图的核心。

软件工程领域内广为流传的一句格言：这个世界上唯一不变的就是需求一直在变。



2.2 需求分析

为什么难以理解问题的需求？主要源于以下几个方面：

□**客户需求的模糊性与复杂性：** 客户往往难以精准、详尽地表述他们对软件系统的需求，因为他们对技术细节不熟悉，或者对自身业务流程、未来愿景的认知尚不清晰。这导致工程师在听取客户需求时，容易产生误解或遗漏关键信息。

□**沟通障碍与语义差异：** 工程师与客户可能来自不同专业背景，使用不同的行业术语和语言风格。即使客户表述清晰，双方也可能因为语义理解上的差异，导致工程师未能准确把握客户需求的真实含义。

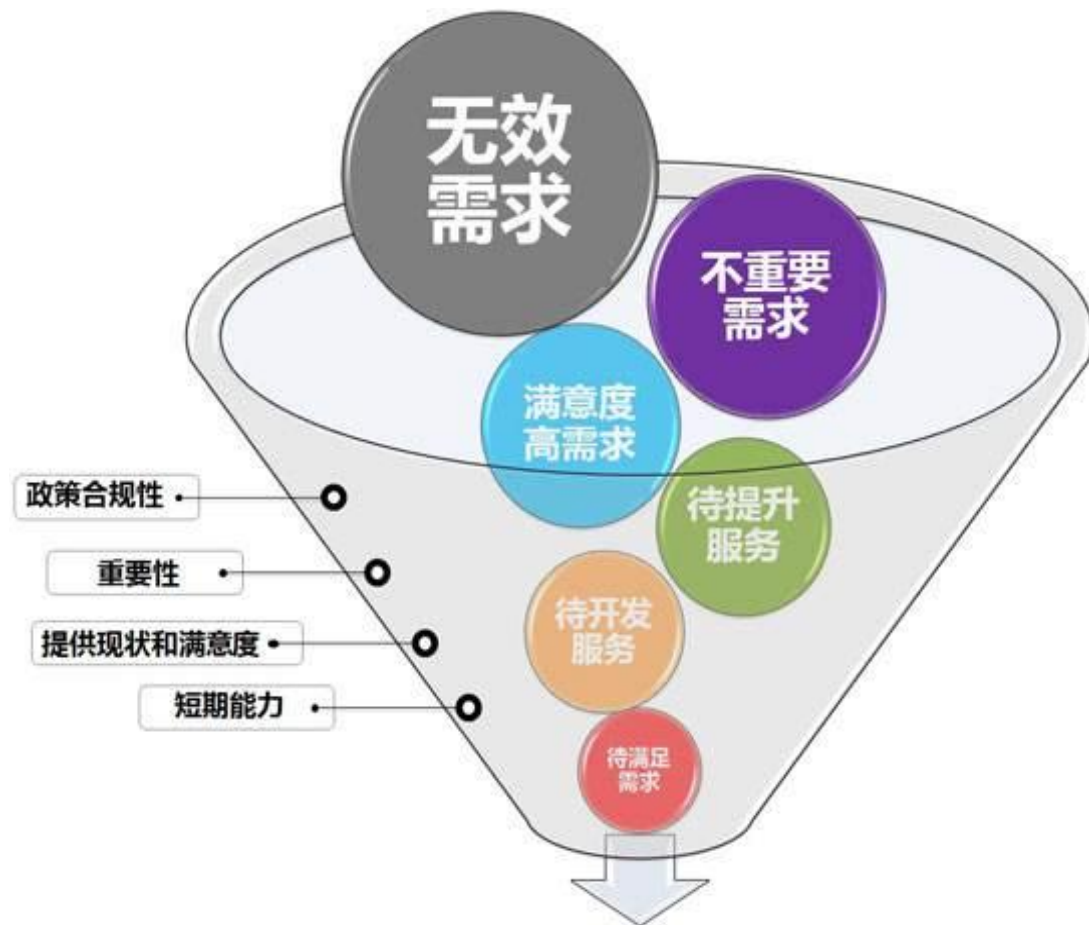
□**需求随时间和环境变化：** 业务环境、市场趋势、竞争态势等**外部因素**，以及客户内部战略调整、组织变革等**内部因素**，都可能导致客户需求发生动态变化。即便前期需求理解准确，后续若未能及时更新，也会造成工程师对客户需求理解的滞后或偏差。

□**需求记录与管理的混乱：** 即使工程师成功捕捉到了客户需求，但如果采用混乱、无序的方式记录和管理这些需求，也会导致需求信息的丢失、混淆或难以追溯。

2.2 需求分析

□需求分析目的：弄清楚到底需要系统“做什么”？

需求分析要求开发人员准确理解客户的需求，进行细致的调查分析，将用户非形式的需求陈述转化为完整的需求定义，再由需求定义转化到相应的形式功能规约（需求规格说明）的过程。



2.2 需求分析

软件需求的关键属性

- **无二义性**。不同的人员对需求的理解应该是一致的。一般情况下，描述需求都使用自然语言，因此容易引起需求理解的二义性。
- **完整性**。每一项需求要完成的任务必须要描述清楚，以便开发人员明白实现这项需求的所有信息，用户能够审查这项需求描述的正确性。
- **正确性**。每项需求都必须准确地反映用户要完成的任务。
- **可行性**。每一个成功的软件系统其解决方案都应该是可行的，可行性体现在技术、经济、操作可行性。
- **必要性**。每项需求都应该是客户所需要的，开发人员不要自作主张添加需求。

这五个关键属性——无二义性、完整性、正确性、可行性和必要性、——是用来确保软件需求的质量、可实施性以及最终软件产品能够满足用户和业务目标的重要标准。

2.2 需求分析

需求分析有五个方面的工作

- **问题识别**：聚焦于发现和理解用户面临的问题、挑战或改进需求，是需求收集和初步筛选的过程。
 - **评价和综合**：对识别出的需求进行深入分析、评估其价值和优先级，并整合成结构化的需求集合。
 - **建模**：通过图形化工具将需求概念转化为易于理解的模型，如UML用例图、数据模型等，便于沟通和理解。
 - **规格说明**：将需求详细、规范地编写成文档，如需求规格说明书，作为后续开发、测试和验收的依据。
 - **评审**：通过多角色参与的评审会议，验证需求文档的正确性、完整性、一致性，确保各方对需求达成共识。
-

2.2 需求分析

SMART原则是一种通用的方法，可以用于任何需要目标设定和规划的场合。在需求分析中，SMART原则可以帮助确保收集和定义的用户需求是明确和可实现的。

- **Specific（具体的）**：需求应清晰、明确，避免模糊或笼统的表述，确保所有参与者对需求的理解一致。
 - **Measurable（可衡量的）**：需求应包含明确的指标或验收标准，以便后续能客观地评估需求是否已得到满足。
 - **Attainable（可实现的）**：需求应在技术、资源、时间等约束条件下是可行的，不应设定超出项目能力范围的需求。
 - **Relevant（相关性的）**：需求应与项目目标、用户需求或业务策略紧密相关，有助于实现项目的整体价值。
 - **Time-bound（有时间限制的）**：需求应设定明确的完成期限或与项目里程碑关联的时间框架，以便合理安排工作进度。
-

2.2 需求分析

需求分析包括**功能需求**和**非功能需求**两个主要方面。

□功能需求：描绘系统应该提供的服务、如何对输入做出反映以及系统在特定条件下的行为的描述。功能需求通常从用户的角度出发，描述了用户希望系统能够完成的工作。

- 例如，在线购物平台的功能需求可能包括用户账户管理、商品浏览、购物车操作、订单处理等

□非功能需求：从各个角度对系统的约束和限制，反映了应用对软件系统质量和特性的额外要求，例如**响应时间、数据精度、可靠性、开发过程的标准**等。

- 例如，在线购物平台的非功能需求可能包括系统响应时间、数据安全性、系统可用性、容错能力等。
-

功能需求案例

智慧农场系统功能需求表

序号	功能需求	描述
1	农田“四情”监测	利用无人机、遥感技术、多光谱、传感器、摄像头、物联网技术，动态监测田间作物的墒情、苗情、虫情和灾情的监测预警系统。用户可以通过手机和电脑随时登录自己的客户端软件，查看田间实时数据并进行管理。
2	农田环境监测	1.利用温湿度传感器监测农田环境的空气温湿度、地表、土壤温湿度等，并对数据分析、运算和存储。2.利用光感和光敏传感器监测、记录农田光照强度。3.部署二氧化碳、氧气浓度传感器能实时监测二氧化碳、氧气的含量。
3	农田小气候观测	农田小气候观测站可以直接测量常规气象因子（大气温度、环境湿度、平均风速风向、瞬时风速风向、降水量、光照时数、太阳直接辐射、土壤温度、土壤热通量、土壤水分和叶面湿度等信息），当信息超出正常范围时，用户可以及时采取防范措施，减轻自然灾害带来的损失，更好的指导农业生产。
4	灌溉设备自动控制	灌溉与农药喷洒采用一套系统，根据作物生长模式，系统可以自动、手动方式进行相应操作。
5	预警系统	采用人工智能技术、大数据技术的农业专家系统进行预测预警，给出农业生产行动方案。
6	农业智能装备与智慧农机	利用无人机来实现药剂、种子等喷洒作业。智慧农机实现大田作业，并实现作业大数据的采集。

功能需求案例

智慧水产养殖系统功能需求表

序号	功能	描述
1	池塘环境监测	配置水温监控、光照监控、水质监控(pH值、溶氧量、氨氮含量等)、品质和药品残留检测、病害监控、气象站、视频监控等监测设备。建设养殖现场无线传输自主网络，实现数据实时采集和自动监控
2	生产过程管理	配置自动增氧、饵料投喂、水体循环与净化、水下机器人等设备，配套实施养殖池塘、车间和网箱标准化改造。通过水产种类、养殖密度、生长阶段、成长状态建立饲喂模型，结合大数据分析优化生产过程管理。开发便携式生产移动管理终端，提升水产养殖的机械化、自动化、智能化水平
3	综合保障管理	研发鱼病远程诊断系统和专家系统，做到疾病早发现，早治疗。开发质量安全可追溯系统，让消费者买到合格的水产品，满足消费者对鲜活产品的要求
4	历史数据分析	系统可以通过不同条件组合查询和对比历史环境数据。系统支持采用数据可视化技术，使用户更直观地看到历史数据曲线

功能需求案例

农产品质量安全追溯系统功能需求表

序号	功能	描述
1	农产品生产主体管理	管理农业生产主体的基本信息和生产基地信息，包括农户、合作社、农场等
2	农产品信息管理	农产品生产主体通过智能终端设备采集农产品土壤耕作、种子选取、发芽、幼苗、开花、结果、收获、储存、运输、销售等各个阶段的信息
3	农作物生产管理	智能采集化肥、农药、饲料等农业投入品的来源、种类、用量等投放信息。建立全生产过程标准化管理档案
4	电子标签系统	在农产品销售时，系统通过编码规则，生成带有产品生产档案信息的电子标签
5	消费者追溯系统	消费者买到带有电子标签的农产品，通过扫描电子标签的一维以查询产品情况，并可以通过系统进行消费维权

第三部分——软件开发阶段

软件设计、编码与测试

3. 开发阶段

□ 软件需求分析一旦完成，就进入开发阶段。需求分析解决“做什么”的问题，
开发阶段解决“怎么做”的问题。

□ **开发阶段有三项技术活动：设计、实现（编码）和测试。**

□ **软件设计是把软件需求变换成软件表示的过程，它主要包含两个阶段：**

- **概要设计（体系结构设计、接口设计和数据设计）**
- **详细设计**

3.1 概要设计

1. 体系结构设计

- 体系结构是具有一定形式的结构化元素，即**构件的集合**。它包括处理构件、数据构件和连接构件。处理构件负责对数据进行加工，数据构件是被加工的数据，连接构件的作用是把体系机构的不同部分组合连接起来。
 - 体系结构设计的主要目的是**开发一个模块化系统**。引入“**分而治之**”的策略，将一个复杂问题分解成可以管理的若干模块。
 - 体系结构设计考虑模块化，但不能过度模块化，不能无限制的划分模块。
 - 开发某个**独立模块**的工作量会随着模块数量的增加而下降。
 - 随着模块数量的增加，集成模块的工作量也会增加。
-

3.1 概要设计

1. 体系结构设计

□进行模块化设计时要考虑模块的独立性，**一般来说，模块独立性越高越好。**

- **提高软件可读性和可理解性：**独立性强的模块使得开发者更容易理解其功能和作用。
- **提高软件可维护性：**当需要修改或更新模块时，独立性高的模块不易受其他模块影响，从而降低维护难度和成本。
- **提高软件可重用性：**独立性强的模块可以更方便地在不同项目或上下文中重复使用。
- **降低模块间的耦合度，**使得系统更加灵活，能够更好地应对需求变更和系统扩展。

□**模块独立性**是通过**耦合性**和**内聚性**来衡量。

3.1 概要设计

1. 体系结构设计

模块的独立性的衡量标准：**耦合性和内聚性**。

□ **耦合(coupling)**：**模块之间**的相对独立性的度量。耦合的强弱取决于模块接口的复杂程度、调用模块的方式以及通过接口的信息。

□ **内聚 (cohesion)**：一个**模块内部**各个元素彼此结合的紧密程度的度量。

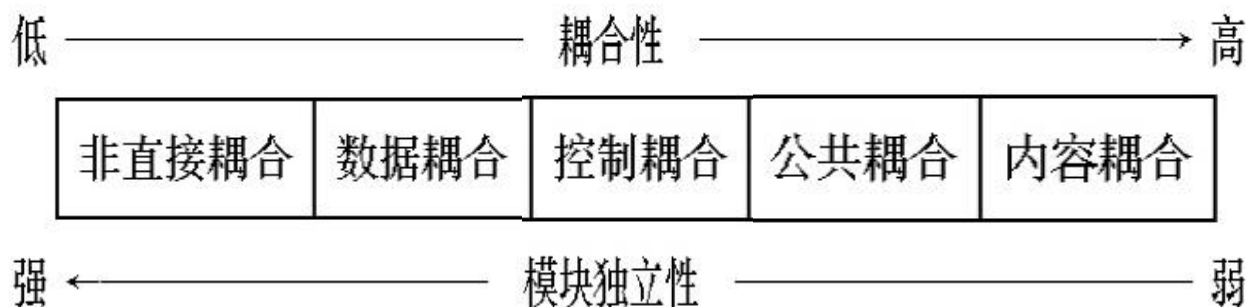
因此，**独立性强的模块**应该是**高内聚低耦合**的模块。

3.1 概要设计

1. 体系结构设计

耦合性： 耦合性是评估不同软件模块之间相互依赖关系紧密程度的标准。它描述了模块间如何相互连接和交互。常见的耦合类型包括：

- **无直接耦合：** 模块间完全独立，没有直接关联。
- **数据耦合：** 模块间通过参数传递数据，但不对对方内部结构有深入了解。
- **控制耦合：** 一个模块通过控制信号（如返回值或状态码）影响另一个模块的行为。
- **公共耦合：** 多个模块共享同一个全局数据区（如全局变量或数据库）。
- **内容耦合：** 模块间直接访问对方内部数据或代码，或一个模块调用另一个模块的大部分功能。



3.1 概要设计

1. 体系结构设计

内聚性：内聚性是用来评价一个模块内部各部分元素（如数据、操作、逻辑等）之间相关性和紧密程度的度量。**高内聚的模块意味着其内部所有元素紧密地围绕一个共同的目标或功能组织在一起，执行的任务单一且明确。**常见的内聚类型包括：

- 巧合内聚：**模块内部元素之间几乎没有关联。
- 逻辑内聚：**模块内部元素基于某种逻辑关系（如相似性）组合在一起。
- 时间内聚：**模块内的操作必须在同一时间段内执行。
- 过程内聚：**模块内的操作按照特定的顺序执行。
- 通信内聚：**模块内所有元素都操作同一数据结构或处理相关的数据项。
- 功能内聚：**模块内部所有元素都紧密服务于单一、明确的功能。（理想情况）

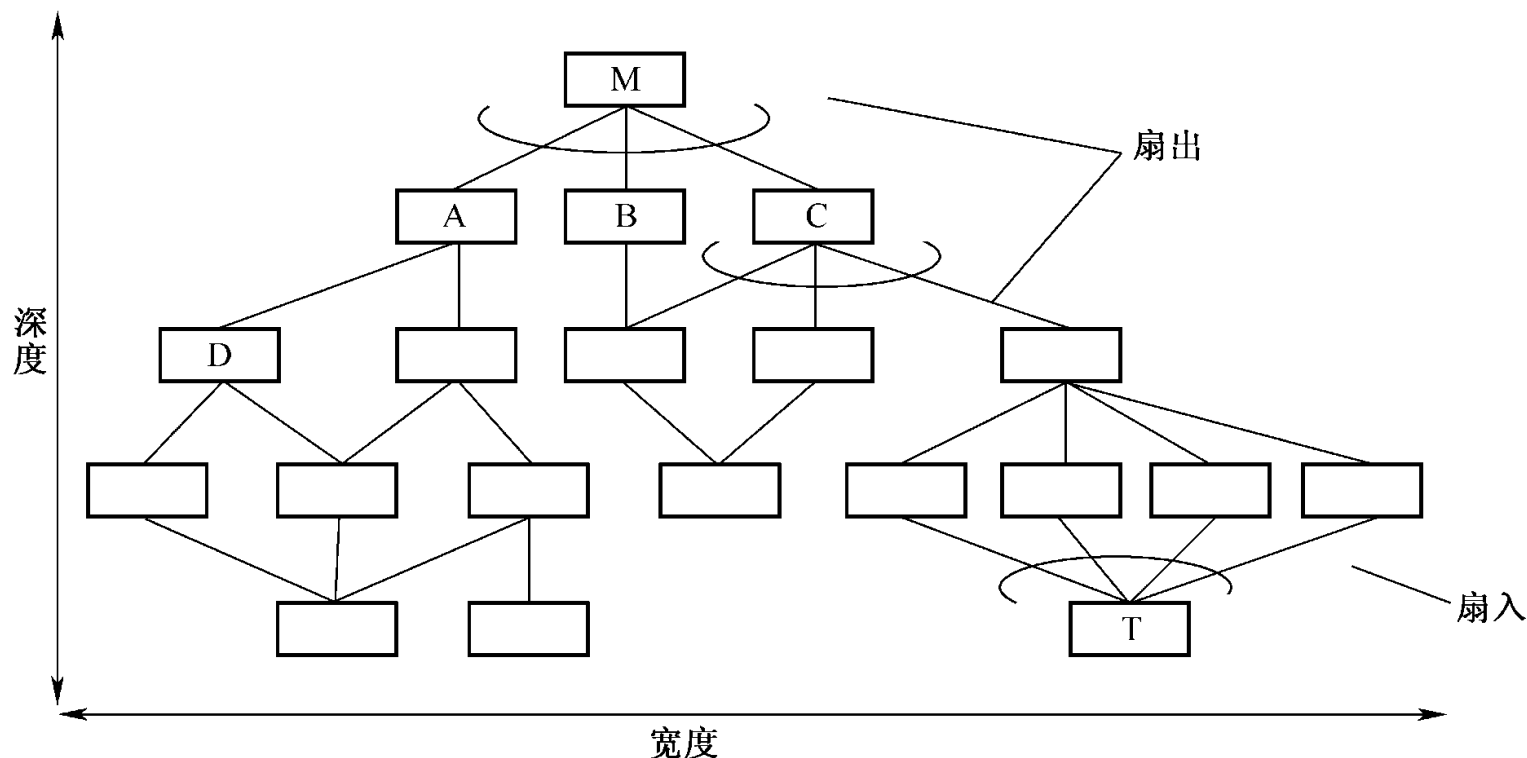


3.1 概要设计

1. 体系结构设计——体系结构图（**分层思想**）

- 体系结构图的**深度**：模块结构的层次数量
- 体系结构图的**宽度**：结构图中同一层模块的最大模块数量
- 模块的**扇出**：指一个模块直接调用的下属模块数量
- 模块的**扇入**：指调用一个给定模块的模块数量。

体系结构图的宽度和深度体现了软件的复杂程度，宽度越宽，深度越深，系统越复杂，功能越多。



填空题

1. 智慧农业系统生命周期由三个阶段的组成，分别为 定义时期，开发时期，维护时期。

2. 接口设计主要包括三个方面，分别为 内部接口设计，外部接口设计，人机接口设计。

3.1 概要设计

2. 接口设计

系统中不同功能层之间的通信规则称为接口。

- 接口设计是软件工程中的一个重要环节，它涉及到软件系统内部各个组件之间以及系统与外部环境之间的交互方式。接口设计的质量直接影响到系统的可维护性、可扩展性和整体性能。
 - 接口设计主要包括三个方面：**内部接口设计、外部接口设计和人机接口设计**
 - **内部接口**：模块或软件构件间的接口
 - **外部接口**：系统与其他软硬件系统之间的接口
 - **人机接口**：软件与用户之间的交互界面
-

3.1 概要设计

2. 接口设计——内部接口设计

- 内部接口设计指的是软件系统**内部各个模块或组件之间的交互接口**。这些接口定义了模块之间的数据流、控制流以及通信协议。
 - 内部接口的设计关键在于确保模块之间的通信尽可能简洁、清晰和高效，同时保持模块的独立性和可重用性。
 - 内部接口设计要点
 - **模块化**：通过模块化设计，减少模块间的直接依赖，提高模块的独立性。
 - **规范接口定义**：每个内部接口应该有明确的定义，包括输入输出参数、数据类型、调用协议等。
 - **抽象和封装**：通过抽象和封装技术，隐藏内部实现细节，提供稳定的接口。
 - **模块间松耦合**：设计接口时应避免紧密耦合，使得模块间的交互最小化。
-

3.1 概要设计

2. 接口设计——外部接口设计

- 外部接口设计涉及软件系统与外部系统或环境之间的交互。这包括与操作系统、数据库、网络服务、硬件设备等的接口。
 - 外部接口设计的目标是确保系统能够高效、安全地与外部世界通信，并满足外部系统的需求。
 - 外部接口设计要点
 - **兼容性**：确保接口与外部系统的标准和协议兼容。
 - **安全性**：设计安全的接口，防止未授权访问和数据泄露。
 - **性能**：优化接口性能，确保数据传输和处理的效率。
 - **文档化**：提供详细的接口文档，方便第三方系统或开发者理解和使用。
-

3.1 概要设计

2. 接口设计——人机接口设计

- 人机接口设计，也称为用户界面（UI）设计，关注软件系统与最终用户之间的交互。包括图形用户界面、命令行界面、触摸屏界面等。
 - 人机接口设计的目标是提供直观、易用、高效的用户体验。
 - 人机接口设计要点
 - **用户为中心**：设计应以用户需求为中心，提供符合用户习惯和期望的界面。
 - **直观性**：界面应直观易懂，用户能够快速理解如何操作。
 - **可用性**：界面应易于操作，减少用户的认知负担。
 - **一致性**：界面风格和操作逻辑应保持一致，避免用户混淆。
 - **可访问性**：考虑到不同能力的用户，设计应具有良好的可访问性。
-

3.1 概要设计

3. 数据设计

- ❑ 数据是软件系统的重要组成部分，在设计阶段必须对要存储的数据及其结构进行设计。
- ❑ 设计者可以采用成熟的关系型数据库，如SQL Server、MySQL、Oracle等。也可以采用非关系型数据库，如文档的数据库MongoDB、键值数据库Redis、图数据库Neo4j和列族数据库Hbase等。



文档数据库	图数据库
 MarkLogic	 Couchbase
 mongoDB	 Neo4j
	 InfiniteGraph The Distributed Graph Database
列族数据库	键值数据库
 Cassandra	 accumulo
 APACHE HBASE	 AEROSPIKE
 HYPERTABLE INC	 redis
	 riak
	 amazon DynamoDB

3.2 详细设计

详细设计是对概要设计活动的延续，其成果将是编程实现的直接依据。

- 概要设计几乎不考虑计算机实现的问题，仍然是把关注的焦点放在系统的逻辑结构上；到了详细设计阶段则更多的是要考虑如何构造用计算机实现的方案。
 - 详细设计阶段的主要任务包括**代码设计**、**数据库设计**、**输入输出设计**、**界面设计**和**算法设计**等。
-

3.2 详细设计

□ 代码设计

- 编码规范、编程风格指南、类/模块设计、数据结构设计、函数设计等。

□ 数据库设计

- 包括数据库模式的设计，如实体关系图、数据表结构、关系模型等。
- 设计数据库的索引策略、存储过程、触发器等，来优化数据的存储和检索效率。
- 定义数据的完整性约束和安全性措施，确保数据的准确性和一致性。

□ 输入输出设计

- 设计用户界面的输入控件，如表单、按钮、下拉列表等，以及它们与后端数据模型的映射关系。
 - 定义数据输入的验证规则和错误提示信息，确保用户输入的数据符合预期格式和业务规则。
 - 设计系统的输出格式，包括数据显示、报表生成、通知消息等，以及它们的样式和布局。
-

3.2 详细设计

□ 界面设计

- 涵盖用户界面的整体布局、色彩搭配、图标设计、字体选用等视觉元素，以及页面跳转、操作反馈、动效等交互细节。

□ 算法设计

- 设计核心算法和数据处理流程，确保软件能够有效地处理数据和执行业务逻辑。
 - 规划业务逻辑的执行顺序和条件分支，处理复杂的业务场景和用户交互。
 - 考虑算法的性能和资源消耗，优化软件的运行效率。
-

3.3 实现阶段

系统设计完成之后，就进入了系统**实现阶段**，即**编码阶段**。

- 编码阶段的任务是根据详细设计阶段提供的设计方案，使用选定的程序设计语言编写实现这些设计的源代码。
- 程序设计语言和程序设计风格会深刻地影响软件的质量和可维护性。为保证编码的质量，程序员必须深刻理解、熟练掌握并正确地运用程序设计语言的特性。此外，还要求源程序具有良好的结构性和良好的程序设计风格。



3.3 实现阶段

1. 程序设计语言的选择

编程语言众多，可以考虑以下几个因素进行选择

- 应用领域
 - 系统用户的要求
 - 编程语言本身的功能
 - 编码和维护成本
 - 编程人员技能
 - 软件的可移植性
 - 稳定性和安全性
-

3.3 实现阶段

2. 程序设计风格

程序设计风格应该尽可能做到源程序文档化、数据说明标准化、语句结构简单化、输入\输出规范化。

□ **标识符命名**：名字应该能反映它所代表的实际意义，有助于对程序功能的理解。常用的标识符命名方法有**大驼峰命名法**（或Pascal命名法）和**小驼峰命名法**（或Camel命名法）。

□ **程序的注释**：正确的注释能够帮助读者理解程序，为测试和维护提供明确的指导。**正规程序文本中，注释行的数量占到整个源程序的1/3到1/2。**

3.3 实现阶段

2. 程序设计风格

- C#、Java等语言编码命名规范对字段、变量、类、方法和属性等指定了统一的命名形式：
 - **类名、方法名和属性名**均使用**大驼峰命名法**。即所有单词连写，每个单词的第一个字母大写，其它字母小写。例如：GetData、HelloWorld等。
 - **变量名、对象名以及方法的参数名**均使用**小驼峰命名法**。即所有单词连写，第一个单词全部小写，其它每个单词的第一个字母大写。例如：userName、userAge等。

3.3 实现阶段

2. 程序设计风格

```
39  /* @覆盖父类或实现接口的onTouchEvent (MotionEvent event) 方法
40  * 方法名: onTouchEvent (MotionEvent event)
41  * 功 能: 触屏事件
42  * 参 数: MotionEvent event - 触屏事件源
43  * 返回值: boolean - 是否撤消该事件
44  * 参 考: @see android.app.Activity#onTouchEvent (android.view.MotionEvent)
45  */
46  @Override
47  public boolean onTouchEvent (MotionEvent event)
48  {
49      //触屏时将连续发生ACTION_DOWN和ACTION_UP两个事件, 这里只处理UP,
50      //如果两个都处理将发生打开两个Activity, 导致打开时闪烁一下
51      if (event.getAction() == MotionEvent.ACTION_UP)
52      {
53          //跳转到搜索条件页面
54          Message message = new Message(); //创建要传递的消息对象
55          message.what = Event.SHOW_HOME; //消息类型
56          Bundle bundle = new Bundle(); //创建消息附加数据对象
57          bundle.putSerializable("source", this); //填充消息附加数据
58          message.setData(bundle); //将附加数据嵌入消息
59          Controller.getHandler().sendMessage (message); //发送消息
60          //System.out.println("event:" + event.getAction());
61      }
62      return super.onTouchEvent (event);
63  }
```

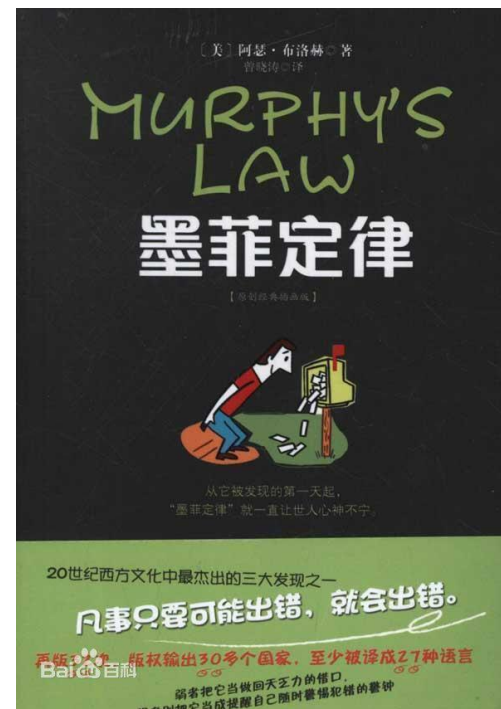
序言性注释

功能性注释

3.4 测试

❑ 测试时保证软件质量的重要手段。测试的目的在于发现错误。

- 一个好的测试用例在于能发现至今未发现的错误;
- 一个成功的测试是发现了至今未发现的错误的测试。

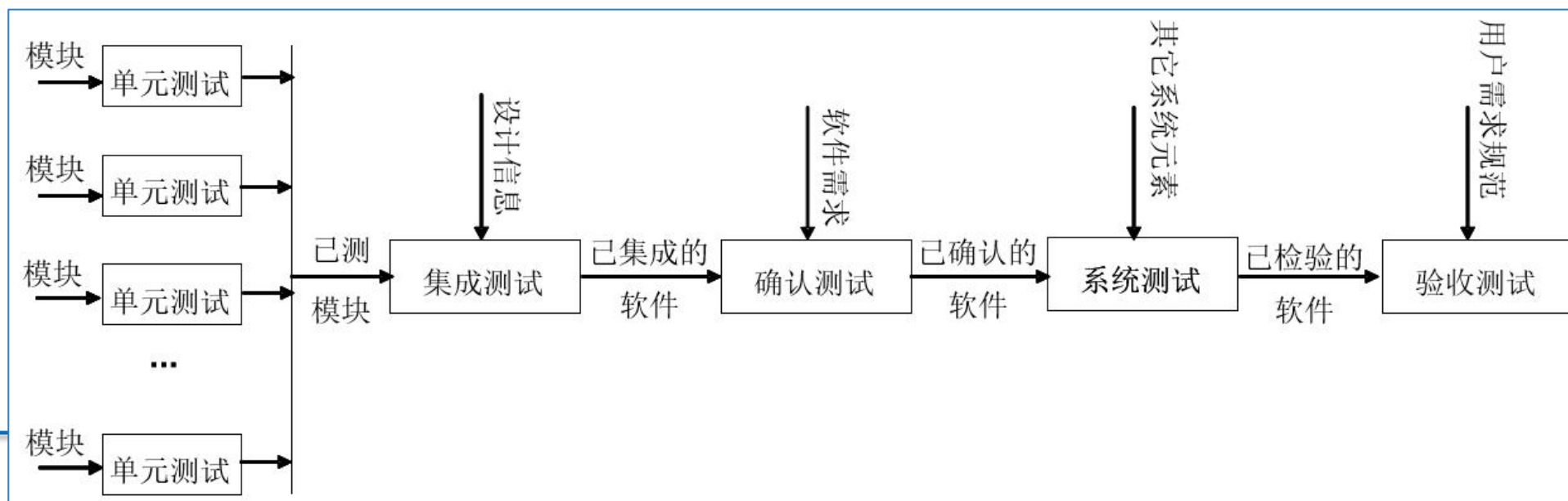


墨菲定律：“如果事情有变坏的可能，不管这种可能性有多小，它总会发生。”

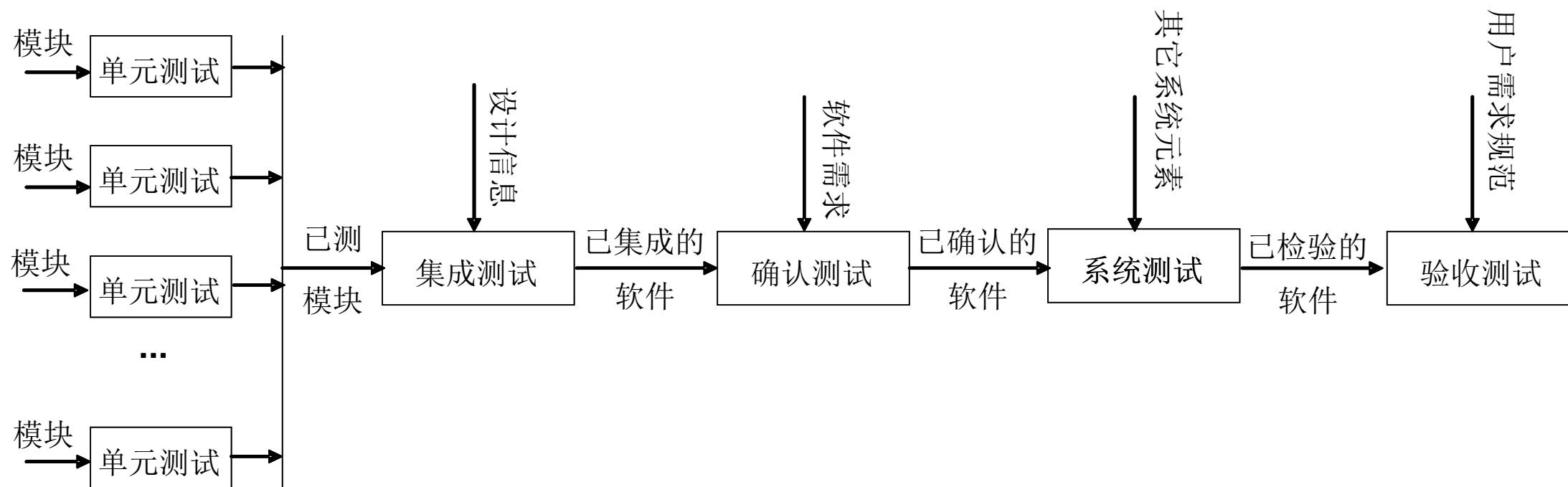
3.4 测试

测试可以分为**单元测试**、**集成测试**、**确认测试**、**系统测试**到**验收测试**。

- **单元测试**：保证每一个模块是按照要求开发的
- **集成测试**：通过单元测试模块按照设计规格顺序组装起来，在组装的过程中检查系统连接中的问题，**重点检查接口和全局数据的错误**。
- **确认测试**：根据需求规格说明书的要求，对必须实现的各项需求进行确认，判定已开发的系统是否符合用户需求。
- **系统测试**：系统测试的任务是将硬件、软件、网络、操作人员看作一个整体、检验它是否有不符合系统说明书的地方。
- **验收测试**：由专业的第三方测试小组承担测试工程，检查系统在实际使用环境中的问题。



3.4 测试



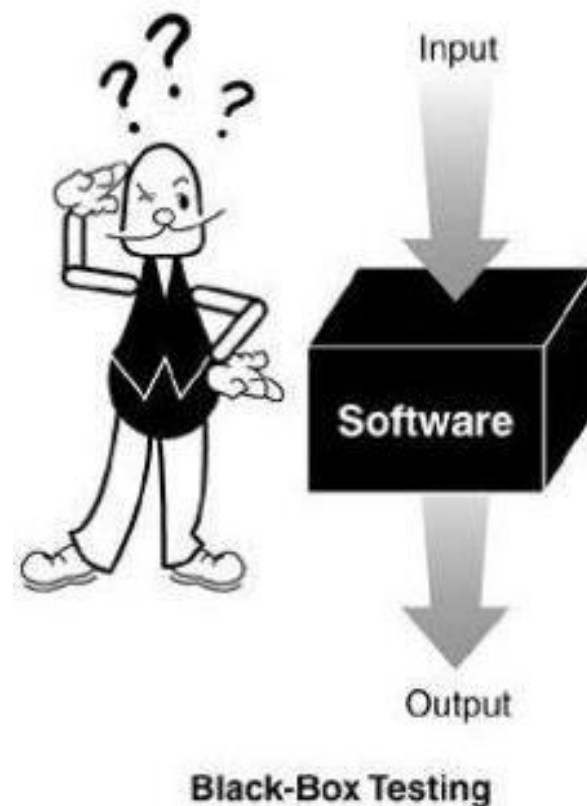
3.4 测试

软件测试中的两种主要方法：**黑盒测试**和**白盒测试**。

□ 黑盒测试（Black-box Testing）是一种**功能测试**方法，测试人员在不了解软件内部结构和内部代码的情况下进行测试。测试人员只关注软件的功能性能，即输入数据后软件是否能输出预期的结果。黑盒测试的主要目的是验证软件的功能需求是否得到满足，以及软件的行为是否符合用户的需求。

□ 黑盒测试的特点

- 测试人员不需要了解程序的内部实现细节。
- 测试基于软件的需求规格说明书。
- 可以发现界面错误、数据结构问题、性能问题等。
- 常用方法有等价类划分，边界值分析，判定表等。



3.4 测试

用户登录系统黑盒测试：用户需要输入用户名和密码，系统规定**用户名必须是有效的电子邮件地址，密码长度必须在6到12个字符之间。**

□ **等价类划分：**是将所有可能的输入数据划分为若干等价类，从而把海量（无限）的测试用例集减得很小

- 有效用户名：符合电子邮件格式的用户名。
- 无效用户名：不符合电子邮件格式的用户名。
- 有效密码：长度在6到12个字符之间的密码。
- 无效密码：长度小于6个字符或大于12个字符的密码。

A screenshot of a user login form. At the top, there is a dark blue header with the text "用户登录" (User Login) in white. Below the header, there are two input fields. The first field is labeled "账号(Account)" (Account) and has a person icon to its left. The second field is labeled "密码(Password)" (Password) and has a lock icon to its left. Both fields are white with a light blue border.

□ **边界值分析：**测试等价类的边界情况，错误往往发生在边界附近。

- **用户名长度边界：**输入长度为0的用户名（无效），输入长度为1的用户名（无效），输入长度为254的用户名（有效）。
- **密码长度边界：**输入长度为5的密码（无效），输入长度为6的密码（有效），输入长度为12的密码（有效），输入长度为13的密码（无效）。

3.4 测试

判定表法：是使用决策表来表示复杂的业务逻辑，以确保所有可能的条件组合都被测试到。在这个决策表中，规则1和规则2分别表示用户名有效和密码长度在6到12个字符之间，这两个条件同时满足时，用户可以成功登录。规则3和规则4分别表示密码长度小于6个字符和大于12个字符，这两个条件满足时，用户登录失败。

基于这个决策表，我们可以设计以下测试用例：

测试用例1：有效用户名，长度为8个字符的密码（预期结果：登录成功）。

测试用例2：无效用户名，长度为5个字符的密码（预期结果：登录失败）。

测试用例3：有效用户名，长度为13个字符的密码（预期结果：登录失败）。

测试用例4：无效用户名，长度为4个字符的密码（预期结果：登录失败）。

通过这些测试用例，我们可以覆盖所有可能的输入组合和边界情况，确保用户登录系统在各种情况下都能按预期工作。

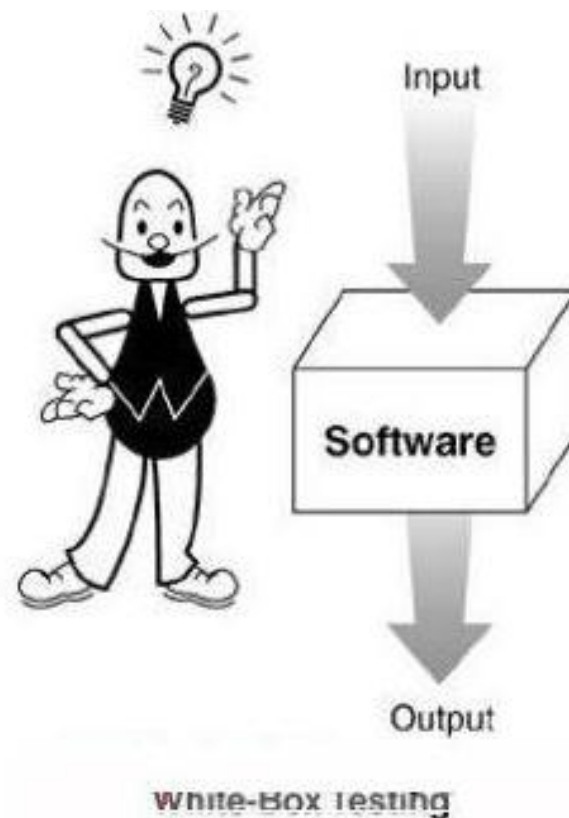
条件\规则	规则1	规则2	规则3	规则4
用户名有效	Y	N	Y	N
密码长度（6-12）	Y	Y	N	N
密码长度（<6）	N	N	Y	N
密码长度（>12）	N	N	N	Y
测试用例	1	2	3	4

3.4 测试

白盒测试 (White-box Testing) 又称为结构测试或透明盒测试，测试人员需要了解软件的内部结构和代码实现。白盒测试关注程序的内部逻辑和代码路径，通过设计测试用例来检查程序中的各种代码路径是否都能按预期工作。

□ 白盒测试的特点

- 测试人员需要了解程序的源代码和内部结构。
- 测试基于软件的内部实现细节。
- 可以发现代码中的逻辑错误、路径问题、循环问题等。
- 常用测试方法包括路径测试、代码覆盖分析、单元测试等。



第四部分——软件维护阶段

运营与维护、评价和审计

4.1 软件维护阶段

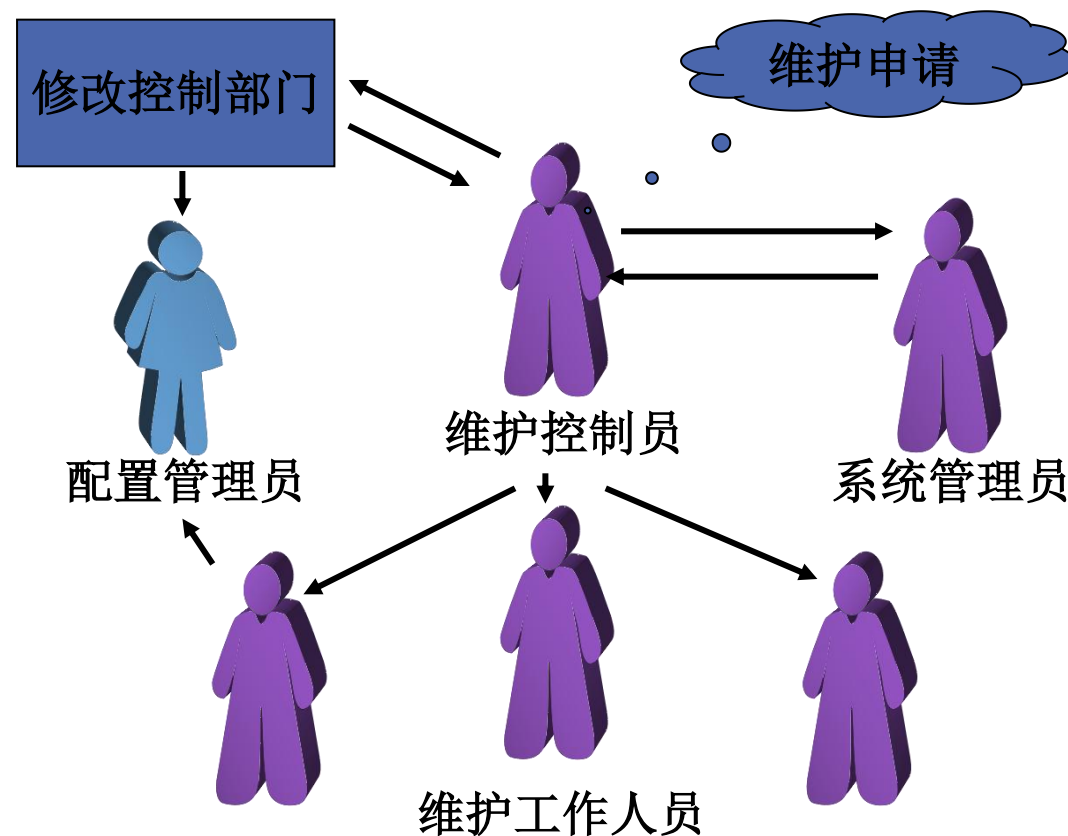
- 运行维护是智慧农业系统已经交付用户使用后，为了使得系统软硬件持久地满足用户的需求，而不断做出修改和完善的过程。
 - 智慧农业系统是一种 “**一次开发，终身维护**” 的产品。
 - 通常由四类维护活动组成：
 - **改正行维护**，也就是诊断和改正系统在使用过程中发现的错误；
 - **适应性维护**，即修改软硬件以适应环境的变化；
 - **完善性维护**，即根据用户的要求改进或扩充系统功能，使其更完善；
 - **预防性维护**，即修改系统为将来的维护活动预先做准备。
-

4.1 软件维护阶段

- **改正性维护**。在系统交付使用后，由于开发时测试不彻底、不完全，有一部分隐藏的误差被带到运行阶段来。这些隐藏下来的误差在某些特定的使用环境下会暴露出来。为了识别和纠正软件误差，改正硬件性能上的缺陷，排除实施中的误使用，应进行的诊断和改正误差的过程就是改正性维护。
- **适应性维护**。随着信息技术的飞速发展,系统运行的外部环境(新的硬件、软件配置)或数据环境(数据库、数据格式、数据输入/输出方式、数据存储介质)可能发生变化，为了使系统适应这种变化而修改的过程称为适应性维护。
- **完善性维护**。在系统的使用过程中，用户往往会对系统提出新的功能与性能要求。为了满足用户的这些要求，需要修改或重新开发系统，以扩充功能，增强性能，改进工作效率，提高系统的可维护性。
- **预防性维护**。为了提高系统的可维护性、可靠性和安全性等，为以后进一步改进系统打下良好基础而做出的维护活动。

4.1 软件维护阶段

- 典型的**维护流程**：从提出维护申请开始，由维护控制员进行管理和调度，经过修改控制部门的审批，配置管理员和系统管理员提供必要的技术支持，最终由维护工作人员完成具体的维护任务，以此确保维护工作的有序进行，并保持系统的稳定和高效运行。
- 一般而言，除大型软件开发机构外，很少设立专门的维护机构。



4.2 评价与审计

- 智慧农业系统科技含量高，投资成本比较大，回收周期长。因此，当系统运行定的时间后，需要对系统的使用情况做出评价与审计，客观、公正、准确地对智慧农业系统使用情况给出结论。
 - 评价与审计通常从系统真实性审计、系统安全性审计和系统绩效审计等三个方面进行。
 - 评价审计的目标主要是考查系统使用的真实性、完整性、可靠性、效果性、效率性、经济性、合法性、安全性等，从而得出智慧农业系统是否具有示范作用，是否值得推广。
-

人生之败，非傲即惰，二者必居其一，
勤则百弊皆除。——《曾国藩家书》

勤

